WHAT BREAKS

Seven assumptions 2026-07-28 stops honoring.

Left: what a client or server built for 2025-11-25 can reasonably believe. Right: what the new revision does instead, and the first place to look in your code.

~~2025-11-25~~ → 2026-07-28

	YOU ASSUMED (2025-11-25)	2026-07-28 DOES THIS
01	Initialize once, and the server remembers your version and capabilities.	**Version and client capabilities travel on every request.** check › how each call builds params._meta
02	The session ID carries what the user selected.	**Protocol sessions are removed. State needs its own handle.** check › explicit project/work handle + access checks
03	The server sends its own approval request to the client and waits.	**The input request comes back inside the original result.** check › retry with inputResponses + requestState
04	Any response means the operation finished.	**A required resultType separates finished from interim.** check › branch on complete / input_required
05	Any HTTP 400 means you reached an older server.	**Modern failures use 400 too: -32020, -32021, -32022.** check › error code + error.data.supported
06	A GET stream or Last-Event-ID recovers lost messages.	**Both removed from Streamable HTTP. Response streams stay.** check › POST response stream, subscriptions, operation recovery
07	Deprecated means it's already gone.	**Deprecated features stay in the spec during their window.** check › per-feature registry entry and notice

ONE SELF-CONTAINED REQUEST

```
POST /mcp
MCP-Protocol-Version: 2026-07-28  ← mirrors _meta
Mcp-Method: tools/call            ← mirrors method
Mcp-Name: publish_note            ← mirrors params.name

{ "jsonrpc": "2.0", "id": "request-a",
  "method": "tools/call",
  "params": {
    "name": "publish_note",
    "arguments": { … },
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/clientCapabilities": { … }
    } } }
```

EASY TO GET WRONG

- `clientInfo` is optional. Version and capabilities are the required pair.
- `Mcp-Name` is required on `tools/call`, `prompts/get` and `resources/read` only.
- Header names are case-insensitive. Method and name values aren't. Decode an encoded name before comparing.
- Servers must implement `server/discover`. Clients may skip it.
- A request ID correlates a response. It isn't an idempotency key, and a retry gets a new one.

READING THE RESPONSE

A 400 is not an era detector.

Falling back to `initialize` on every 400 hides real defects. Read the code first.

YOU SEE	IT MEANS	DO THIS
HTTP 400 · `-32020`	A header is missing, malformed, or disagrees with the body it mirrors	Compare the exact headers with method, name and version in the body
HTTP 400 · `-32021`	The call needs a client capability you didn't declare	Read `data.requiredCapabilities`; declare it if you support it
HTTP 400 · `-32022`	The requested version isn't supported	Pick one from `error.data.supported` that you also support
HTTP 404 · `-32601`	Unknown method	Check what this server supports
HTTP 404, no MCP body	Wrong endpoint, or an intermediary answered	Destination and routing logs
`-32700` / `-32600` / `-32602`	Parse error / invalid envelope / invalid params	Original bytes, object shape, the method's contract
`result.isError: true`	The tool ran and failed	The tool result and your application's evidence
No response at all	Unknown. The write may or may not have happened	Application commit evidence, never the status alone

When the server needs an answer mid-call

No sessions means no callback over an open connection. The server returns an interim result, and the client calls again with the answer. The approval rule itself doesn't change.

1 · CLIENT CALLS
tools/call with its arguments and `_meta`, as usual.

→

2 · SERVER ASKS
Returns `resultType: "input_required"`, the `inputRequests` and an opaque `requestState`.

→

3 · CLIENT RETRIES
Same call, same arguments, a new request id, plus `requestState` and `inputResponses`.

SERVER · INTERIM RESULT

```
{ "resultType": "input_required",
  "requestState": "OPAQUE_STATE_FROM_RESULT",
  "inputRequests": {
    "approval": {
      "method": "elicitation/create",
      "params": { "mode": "form", … }
} } }
```

CLIENT · RETRY, INSIDE PARAMS

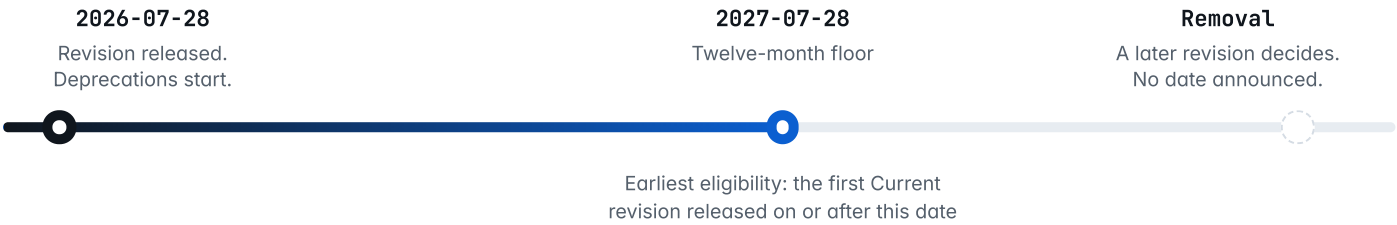
```
"name": "publish_note",
"arguments": { … unchanged … },
"requestState": "OPAQUE_STATE_FROM_RESULT",
"inputResponses": {
  "approval": { "action": "accept",
    "content": { "confirm": true } } },
"_meta": { … as before … }
```

Echo requestState byte for byte. Don't decode it, edit it or reuse it for another operation. `inputResponses` sits beside `arguments`; it isn't a new tool argument. A result from an earlier server with no `resultType` is treated as complete, not as malformed.

THE DEPRECATION CLOCK

Deprecated is not removed.

Four features are deprecated in 2026-07-28 and still in the spec. The registry gives the earliest point they become eligible for removal. That isn't a shutdown date.



FEATURE	STATUS	MOVE TOWARD	EARLIEST REMOVAL
Roots	Deprecated	Explicit tool arguments, resource URIs or server configuration	First Current revision ≥ 2027-07-28
Sampling	Deprecated	A direct provider integration with its own tested policy and output checks	First Current revision ≥ 2027-07-28
Logging	Deprecated	stderr for stdio, or your observability integration	First Current revision ≥ 2027-07-28
Dynamic Client Registration	Deprecated, compatibility kept	Client ID Metadata Documents where supported	First Current revision ≥ 2027-07-28
`includeContext` **thisServer / allServers**	Deprecated	Omit it, or use `none`	Follows Sampling
Historical HTTP+SSE transport	Transition provisions	Streamable HTTP	Three months after SEP-2596 is Final

ALREADY REMOVED

`logging/setLevel` and `notifications/roots/list_changed`. Log level is set per request now, and protocol log messages need that opt-in. Don't confuse these with the broader deprecations.

STILL THERE

SSE response streams on a POST, and the SSE stream of `subscriptions/listen`. Only the standalone GET stream and `Last-Event-ID` resumability left Streamable HTTP.

Two more clocks run beside this one. **Expedited removal**: the lifecycle policy allows an approved exception with at least ninety days between deprecation and earliest removal. **Your SDK** may drop a revision on its own timeline, separate from the spec.

FILL ONE PER FEATURE YOU DEPEND ON · LEAVE UNKNOWN DATES EMPTY

Feature _____ Status · source · date checked _____

Revision that deprecated it _____ Earliest eligibility rule _____

Approved exception, if any _____ Announced removal, if any _____

SDK support policy _____ Our internal deadline · owner _____

STAGED ROLLOUT

Switch without losing work in flight.

Each stage has a record you keep and a condition that stops the rollout. Ticking a box is not evidence; the record is.

	STAGE	KEEP A RECORD OF	STOP IF
☐	Inventory	Actual client builds, transport and revision, and the evidence	Any compatibility assumption is unknown
☐	Baseline	Accepted and refused business behavior, with exact effects	Current behavior can't be reproduced
☐	Conversion	Explicit selection handles, input retry, identity binding	A new call depends on old process memory
☐	Candidate	Acceptance per client cohort, and the state it leaves	Result mismatch or an unauthorized effect
☐	Pending work	Count and disposition of live earlier callbacks	Old operations are unresolved
☐	Rollback	The earlier reader recovers candidate-written records	Repeated effect, unreadable state, lost work
☐	Source review	Dated per-feature notice and eligibility rule	Stale or contradictory records
☐	Approval	Named operator, change window, tested restore	No owner, or a restore nobody has run

Four traps, and the answer to each

A call gets HTTP 400 with `-32020`. Should the client fall back to initialize?

No. That's a modern header mismatch. Fix the request.

Callbacks are gone. Does the server still need approval before it writes?

Yes. The business rule stays. Deliver the question through input_required.

Should we schedule Sampling to switch off on 2027-07-28?

No. That's earliest eligibility, not a removal date. Keep a dated record.

We stream SSE responses today. Does all SSE code have to go?

No. Response streams stay. Only the GET stream and resume left.

Code rollback isn't transaction rollback. If a write committed before a bad response went out, the old path will find it. Check what was written before you repeat an operation whose answer was lost.



Do this with a lab: MCP Production

Module 9 runs this migration on fixtures for both revisions, with a promotion gate that refuses the rollout until the evidence is there. 57 lessons, 14 local Docker labs, Python.

runsclean.com/courses/mcp-production

Sources, checked 10 September 2026: changelog versioning streamable-http mrtr deprecated feature-lifecycle